

PROJEKTBESEKRIVNING

CRUD med en resurs

För 3,5 hp i kursen Dynamiska Webbapplikationer ska två labbar och ett onlineprov utföras. Detta är den första labben. Den utförs i labbgrupper, enligt de som finns på Learn (markerade Projektgrupp XX).

Läs igenom instruktioner för förberedelser, uppgifter och inlämning noggrant och tveka inte att höra av er till mie@du.se om det är några frågor.

Lycka till!

Innehåll

Förberedelser	4
Kravspecifikation	5
Tekniska krav	5
Gränssnitt	5
1. Styling.....	5
2. Lista	6
3. Formulär	6
4. Knappar/ikoner	6
5. Meddelande.....	6
6. Navigation – valfritt!.....	6
Tekniska detaljer.....	7
Överkurs.....	7
Backend.....	7
1. API-endpoints.....	7
2. Gemensamt Funktionalitet i Callbackfunktionen hos alla API-endpointss.....	7
3. Databaskommunikation	8
4. Databasen	9
Frontend.....	10
1. Generellt	10
2. Visa alla (R i CRUD).....	10
3. Möjliggöra uppdatering av resurs.....	11
4. borttagning av en resurs. (D i CRUD).....	11
5. Formuläret	12
6. Skicka formulär (C och U i CRUD).....	12
7. Meddelanderuta	12
Inlämning.....	13
Kod	13
Loggar	13
Redovisning.....	14
Upplägg	14
Presentation.....	14
1. Upplägg och tidsåtgång	14

2. Innehåll.....	14
Notering rörande Live Server	15

Förberedelser

1. Skapa ett lokalt repository på någon av er dator, exempelvis enligt instruktioner i [Git, GitHub, & GitHub Desktop for beginners](#).
2. Döp det till **gik339-[gruppnummer]-projekt**.
 - a. Ta inte med punkten ovan. Det blir fel på repositoryt om den slutar med en punkt.
 - b. Om ni följer videon ovan kommer detta skapa en mapp av samma namn på en given position (angivet i fältet **Local path** i GitHub for Desktop).
3. Öppna mappen för repositoryt i VS Code.
4. Utför labben enligt instruktioner
 - a. **OBS!** när ni gjort förändringar och ska [publicera ert repository till GitHub](#), se till att inställningen **Keep this code private** *inte* är markerad.
5. Om gruppen vill arbeta från olika datorer, se till att alla jobbar utifrån samma repository på GitHub.
 - a. Alltså, bara en ska skapa ett repo de andra ska bara hämta [det befintliga på GitHub](#) och ladda ner det lokalt till sin dator.
 - b. Det ska alltså bara finnas ett (**1**) repository som ni ska arbeta mot, och vars länk ni ska lämna in enligt avsnittet Inlämning.
6. Ert repository *ska innehålla* en fil vid namn **.gitignore** innehållande texten **node_modules/** för att *exkludera* mappen **node_modules** från ert repository.

Kravspecifikation

Ni ska skapa en webbsida där man kan utföra **CRUD-funktionalitet** för en viss **resurs**. Det kan vara vad som helst – generella produkter, filmer, bilar eller något annat. Det får dock inte vara något stötande och det får inte vara användare (**users**) eller uppgifter (**tasks**). Sådana har vi jobbat tillräckligt med under kursen.

Eftersom jag inte vet vad er webbsida kommer att handla om, kommer jag vidare i dokumentet använda begreppet **resursen** för att prata om det som sidan ska handla om.

Webbsidan kommer, när den är färdig, att likna **användarregistrerings-applikationen** såsom den såg ut efter den tillgängliga **inspelade tutorial** i fem delar som finns i kursrummet.

Det kommer dock att finnas detaljer och krav som ni inte ännu har sett under kursens gång och som ni förväntas lösa på egen hand.

Tekniska krav

1. Datalagring ska ske i en SQL-databas, förslagsvis SQLite
 - a. **Notera:** Detta kan behöva konfiguration av era datorer och VS Code, ifall ni inte arbetat med Lektion 4 eller Labb 3.
2. Backend ska skrivas i en Node.js-miljö med stöd av Express.
 - a. **Notera:** Detta kan behöva installation av Node.js på era datorer, ifall ni inte arbetat med Lektion 4 eller Labb 3.
3. Backend och frontend ska kommunicera med varandra genom ett **API**.
4. Inga externa JavaScript-ramverk (såsom **React** eller **Vue**) får användas i frontend.
5. Endast **en (1)** HTML-sida ska användas. All information där ska skapas och ändras dynamiskt utifrån hur data hämtas och förändras.

Gränssnitt

Nedanstående delar **ska** uppfyllas hos er webbsidas gränssnitt. Delar kommer att skapas dynamiskt i JavaScript-kod, andra delar ska skrivas direkt i HTML-filen från början. Detaljer rörande dessa element följer nedan.

1. STYLING

Styling av webbsidan ska ske med hjälp av ett **CSS-ramverk**. Det kan vara [Bootstrap](#), [Tailwind](#) eller något annat ramverk. Webbsidan måste vara **responsiv**, vilket ska lösas med hjälp koncept såsom [Grid](#) (länken är till Bootstrap, men konceptet finns i de flesta CSS-ramverk). Egen CSS-styling får användas, exempelvis rörande detaljer såsom färg, men i huvudsak ska ett ramverk användas.

Regler rörande styling ska tillämpas på såväl element i som från början finns i HTML-filen, som **dynamiskt genererade** HTML-element som läggs till via JavaScript.

2. LISTA

Det ska finnas en lista som **visar alla** lagrade poster för er valda **resurs**, hämtade från ert backend. Listan ska alltid innehålla **aktuell information** från servern. Listan får se ut hur som helst. Respektive **resurs** kan presenteras exempelvis i en lista eller i s.k. [Cards](#) och i vilken ordning ni vill.

3. FORMULÄR

Det ska finnas ett formulär för att möjliggöra **inmatning** av data för **all information** (utom **id**) hos er **resurs**. Hur många och av vilken typ inmatningsfälten är bestäms av hur ni utformar er databas. Formuläret ska ha som minst ha en knapp för att skicka (**submit**) formuläret, men kan också ha knappar för att exempelvis rensa fält.

4. KNAPPAR/IKONER

Knappar för att möjliggöra **borttagning** och **ändring** av **resurs** ska finnas i gränssnittet, förslagsvis intill respektive **resurs**. Ni väljer själva om detta är **knappar** eller **ikoner** i gränssnittet.

5. MEDDELANDE

När en ny resurs skapats, tagits bort eller uppdaterats ska ett meddelande, exempelvis i form av en [modal ruta](#) visas som feedback till användaren.

6. NAVIGATION – VALFRITT!

Ni får lägga till något slags navigation om det önskas, men er webbsida ska vara en s.k. Single Page Application (SPA). Ni behöver alltså se till att allt innehåll finns i en och **samma index.html**. Alltså ingen navigation till någon annan **.html**-fil, utan endast navigation av den typ att något på sidan **döljs/visas** beroende på interaktion.

Tekniska detaljer

Nedan listas detaljer rörande kraven. Dessa är *minimum*. Det är säkerligen möjligt att lösa vissa av nedanstående krav på flera sätt än eventuella exempel som jag ger. Känn er fria att lösa uppgifterna hur ni vill, men stäm av så att alla faktiska krav uppfylls.

Överkurs

Om ni tycker att denna projektuppgift är väldigt enkel, bygg ut den efter behag. Ni får fritt lägga till eller förändra kringliggande funktionalitet och kod, men pyssla med sådant först *efter* att ni har minimumkraven på plats. Jag kommer inte att specificera några speciella extrauppgifter, men det kan röra sig om fler funktioner i gränssnittet, mer avancerad styling, frontend/backend-validering av data, eller fler tabeller och relation mellan flera resurser.

Backend

1. API-ENDPOINTS

1. Det ska finnas **lämpliga API-endpoints** för att möjliggöra **alla funktioner** i frontend.
2. API-endpoints ska utformas och namnges enligt nedan (**RESTful-standard**)
3. De API-endpoints som ska finnas är:
 - hämta alla (**GET /resurs**)
 - uppdatera (**PUT /resurs**)
 - skapa (**POST /resurs**)
 - ta bort (**DELETE /resurs/:id**).
4. Icke obligatoriskt API- endpoint:
 - hämta en **resurs** (**GET /resurs/:id**). Huruvida den behövs eller inte beror på hur ni utformar ert frontend.
5. Respektive endpoint hanteras genom **express** och dess **.get**, **.post**, **.put**, **.delete**-funktioner respektive. Dessa tar alla en **callbackfunktion** där funktionaliteten för era respektive endpoints ska skrivas.
 - **Exempel:** en endpoint för att hantera **GET /resurs** kan se ut såhär:
`server.get(/resurs, (req, res) => { //callbackfunktion för att hantera förfrågan })`.

2. GEMENSAMT FUNKTIONALITET I CALLBACKFUNKTIONEN HOS ALLA API-ENDPOINTSS

1. **request**-objekt ska läsas **för att hämta relevant data som skickas från frontend**. Dessa kan finnas i exempelvis **body** eller **params** (data i url:en), beroende på hur förfrågan från frontend ser ut.
2. **response**-objektet **ska användas för att skicka svar från servern**.

3. Om ni vill kan även lämpliga **meddelanden** skickas med **response**-objektet (tillsammans med andra eventuella data) för att ge feedback till användaren.
4. Om det någonstans **uppstod ett fel**, ska status **500** skickas, tillsammans med ett felmeddelande i **response**-objektet.
 - Till detta kan ni exempelvis använda [try-catch](#), eller på andra sätt fånga upp fel.
 - Node-modulen **sqlite3** har också inbyggda funktioner för felhantering som kan nyttjas.

3. DATABASKOMMUNIKATION

1. Koppling till databasen görs i callbackfunktionen till **respektive endpoint**.
2. Hur SQL-frågorna för respektive funktion ska se ut får ni själva ta reda på. Länkar till respektive SQL-fråga finns nedan.
 - Instruktioner rörande SQL specifikt för SQLite i Node.js finns på denna länk: <https://www.sqlitetutorial.net/sqlite-nodejs>.
3. Vid respektive endpoint **ska förfrågan resultera i databas-funktionalitet enligt nedan**:
 - Vid **POST**-förfrågan ska **en ny rad** sparas (**INSERT**) i databastabellen för er **resurs** baserat på innehåll i **request**-objektets **body**.
 - Vid **PUT**-förfrågan ska en **befintlig rad** uppdateras (**UPDATE**) i databastabellen för er **resurs** baserat på innehåll i **request**-objektets **body**.
 - För att veta vilken resurs som ska uppdateras ska egenskapen **id** i **request**-objektets **body** användas som **WHERE**-villkor i SQL-frågan.
 - Vid **DELETE**-förfrågan (med **id** som parameter i url:en) ska en **befintlig rad** tas bort (**DELETE**) ur databastabellen för er **resurs**.
 - För att veta vilken **resurs** som ska tas bort ska egenskapen **id** i **request**-objektets egenskap **params** användas som **WHERE**-villkor i SQL-frågan.
 - Vid **GET**-förfrågan (utan **id** som parameter i url:en) ska **samtliga rader** hämtas (**SELECT**) ur databastabellen för er **resurs**.
 - Vid **GET**-förfrågan (med **id** som parameter i url:en) ska **en rad** hämtas upp (**SELECT**) ur databastabellen för er **resurs**.
 - För att veta vilken **resurs** som ska hämtas ska egenskapen **id** i **request**-objektets egenskap **params** användas som **WHERE**-villkor i SQL-frågan.

4. DATABASEN

1. En **SQL-databas ska användas**, förslagsvis SQLite.
2. Motsvarande **npm-paket** ska användas för att kunna jobba med databasen i Node.js (**sqlite3** för att arbeta med en SQLite-databas).
3. Databasen **ska innehålla en (1) tabell**, motsvarande den **resurs** du arbetar med.
4. **Minst en kolumn** ska på sikt styra någon **grafisk aspekt** i frontend, såsom storlek, färg eller form på det element som ska representera den enskilda **resursen**.^[1]
5. I övrigt får tabellen få ha **vilka och hur många kolumner ni vill** (inom någon av rimlighet, förstås).
 - Kolumnerna kan döpas på samma sätt som t.ex. **id**- eller **name**-attributen i formulärets fält i frontend. Det är inte helt enligt korrekt namnstandard, men det förenklar mycket om JSON-objektets nycklar motsvarar tabellens kolumner.
6. Ni **bestämmer själva** vilka **datatyper** och andra eventuella restriktioner som finns hos kolumnerna.
 - Tabellen **ska** dock ha en **id**-kolumn, vars värde automatiskt sätts och ökar automatiskt för varje rad som läggs till i tabellen (**AUTOINCREMENT**). Ett **id** ska alltså aldrig skickas med en **INSERT**-query till databasen.

¹ Ett exempel kan vara om resursen är **bilar** kan bilens **färg** påverka utseendet i gränssnittet, men det behöver inte vara så konkret heller. Man kan ha att vissa kategorier representeras av ett visst utseende, t.ex. åldersgrupper har olika färger.

Frontend

1. GENERELLT

Samtliga anrop för att **hämta eller förändra data** ska göras via **fetch()**-anrop till ett **backend som ni själva skapat**, via ett **API** som ni satt upp för att möjliggöra kommunikation till ert backend (se [Backend - API-endpoints](#)).

Innehållet på sidan ska alltid vara aktuellt och uppdateras (**utan sidomladdning [2]**) vid förändring av data.

Nedan listas detaljer rörande kraven. Dessa är minimum. Det är säkerligen möjligt att lösa vissa av nedanstående krav på flera sätt än eventuella exempel som jag ger. Känn er fria att lösa uppgifterna hur ni vill, men stäm av så att alla faktiska krav uppfylls.

2. VISA ALLA (R I CRUD)

1. En lista av **något slag** ska presentera **samtliga rader** ur databastabellen som lagrar **resursen**.
 - Listan kan vara av typen **ul** med tillhörande **li**-element för varje enskild **resurs**, men andra lämpliga HTML-element får också användas. [\[3\]](#)
2. Listan får **inte finnas i HTML-koden från början**.
3. Listans innehåll ska hämtas (**GET /resurs**) från ert backend via ert API.
4. När svar kommit från ert backend ska **HTML skapas och läggas till i DOM-trädet** för att **synas på webbsidan**.
 - Först nu kommer alltså listan att läggas till i DOM-trädet, eftersom den inte ska finnas där från början.
5. **Valfritt antal egenskaper** hos **resursen** ska synas i respektive listelement
 - Något eller några egenskaper ska dock påverka någon **designaspekt** hos listelementet.
 - Egenskapen **id** behövs också någonstans, men det ska **inte synas** i listan. Ett alternativ är att id-/klass-/[data](#)-attribut i en HTML-tagga för att lagra **id**:t på något sätt.
6. Listan ska **alltid ha aktuell data**, så den behöver uppdateras vid förändring.
7. Ni får, men behöver inte, hantera klick på respektive rad/objekt för att exempelvis visa en detaljvy av **resursen**.

² Förtydligande: Omladdning av sidan kan ske på olika sätt exempelvis klick på **refresh**-knappen i webbläsaren, användning av **window.location.reload**, eller att Live Server uppdaterar sidan (en inställning hos Live Server som ni behöver vara uppmärksamma på). En ny förfrågan (**fetch()**) mot ert backend behöver göras **efter att något har förändrats i databasen** för att detta krav ska vara uppfyllt.

³ Jag kommer att dock vidare använda begreppen **lista** och **listelement** för att prata om denna samling och dess enskilda element.

- Knappar/ikoner för att **ta bort** och **ändra resurs** kan med fördel också presenteras i respektive listelement intill övrig information om **resursen**. De **behöver finnas någonstans**, och denna plats vore lämplig.

3. MÖJLIGGÖRA UPPDATERING AV RESURS

- Eventlyssnare** ska finnas som lyssnar efter **klick på knapp/ikon** för att **ändra resurs** (se [Gränssnitt - Knappar/ikoner](#)).
- Vid klick på knappen ska **befintliga uppgifter** om vald **resurs presenteras i formulärets fält**.
 - Notera:** Klick på denna knapp ska **inte skicka** en faktisk förfrågan till servern om att **uppdatera** uppgifter, utan endast **fylla i formulärets fält** med befintlig data så att man kan ändra dem där och sedan skicka data **via formuläret**.
 - Notera:** Om inte alla egenskaper finns i listan redan behöver en **ny förfrågan** till API:et göras så att **samtliga fält i formuläret** kan fyllas på med data.
 - (**GET /resurs/:id** är en lösning. Det måste då förstås finnas en sådan route i ert API.
 - Formulärets fält kan då fyllas på **först när svar från backend** mottagits.
- Någonstans behöver **id** sparas, för den resurs vars data **för närvarande finns i fälten**. Det behövs i [Punkt 6 - Skicka formulär \(C och U i CRUD\)](#).
 - Exempel på lösning kan vara användning av **localStorage** eller att lagra **id** någonstans (**osynligt**) i anslutning till formuläret.
 - Egenskapen **id** ska **inte** synas någonstans i gränssnittet.
- Någonstans måste alltså **resursens id** hämtas/finnas tillgängligt i eventlyssnaren.

4. BORTTAGNING AV EN RESURS. (D I CRUD)

- Eventlyssnare** ska finnas som lyssnar efter **klick på knapp/ikon** för att **ta bort resurs** (se [Gränssnitt - Knappar/ikoner](#)).
- Vid klick på knappen ska **klickeventet fångas** upp och göra en förfrågan med metoden **DELETE** till ert API.
- Någonstans måste alltså **resursens id** hämtas/finnas tillgängligt i eventlyssnaren, så att man kan veta vilket **id** som ska skickas med **DELETE**-förfrågan till API:et.
- När svar från servern har mottagits ska ett meddelande visas, se [Punkt 7 - Meddelanderuta](#).
- När svar från servern har mottagits ska listan uppdateras dynamiskt, **utan** sidomladdning.

5. FORMULÄRET

1. Ett formulär med **fält som representerar de fält som er resurs har i databasen** ska finnas på webbsidan.
 - Alla fält utom **id** ska finnas representerade.
 - Antalet fält styrs alltså av hur ni utformat er databastabell.
2. Använd **lämpliga typer** av HTML-inputelement beroende på **datatyp i databasen**. Om något lagras som en **INTEGER** i databasen är det alltså vettigt att använda ett **input-fält** av typen **number** i formuläret.
3. Ni får validera fältets data genom HTML-validering eller JavaScriptfunktioner, men ni behöver inte.
4. Formuläret måste som minst ha en **knapp för att skicka in** (typen **submit**), men kan även ha knappar för att exempelvis rensa fält, om man vill.

6. SKICKA FORMULÄR (C OCH U I CRUD)

1. Funktion (**eventlyssnare**) ska finnas för att hantera formulärets **submit**-event.
 - Fundera här på hur ni kan undvika duplicering av kod. Mycket av funktionaliteten för att skapa och för att uppdatera **resursen** är lika och koden kan skrivas på mer eller mindre effektiva sätt för att lösa detta.
2. Innehållet i formuläret (samtliga uppgifter hos **resursen**) ska skickas i **JSON-format** via **request**-objektets **body**.
3. När svar från servern har mottagits ska ett meddelande visas, se [Punkt 7 - Meddelanderuta](#).
4. När svar från servern har mottagits ska listan uppdateras dynamiskt, **utan** sidomladdning.
 - Se till att Live Server inte råkar ladda om er sida när ni testar detta, se [Notering rörande Live Server](#).

7. MEDDELANDERUTA

1. Det ska finnas en ruta som ska **visa feedback till användaren** när något händer med **resursen**.
 - a. Den kan med fördel vara en [Modal från Bootstrap](#), eller liknande.
2. Meddelanderutan ska visas när en **resurs** har **skapats, uppdaterats** eller **tagits bort** och svar från servern mottagits.
3. Texten/meddelandet kan komma från servern via ett **response**-objekt, men kan också vara något annat som passar den förändring som skett.
4. Ni får gärna använda meddelanderutan i andra sammanhang också, t.ex. för fältvalidering eller annan användarfeedback.

Inlämning

Senast 23:59 onsdag den 14 januari ska nedanstående lämnas in i kursrummet.

Kod

- Lämna in länk till GitHub-repository som skapades och lades upp på GitHub under [förberedelserna](#).
 - Länken ska se ut något i stil med: `https://github.com/[ditt-github-användarnamn]/gik339-[gruppnummer]-projekt`.
- **Observera!**
 - Lämna **inte** in en zip:ad mapp med er kod!
 - Ladda **inte** upp mappen **node_modules** till GitHub (se [Förberedelser, punkt 6](#)).

Loggar

Om generativ AI har använts ska ett dokument innehållande dokumentation om hur den använts inkluderas i inlämningen. Detta kan röra sig om exempelvis:

- loggar från **ChatGPT** eller andra chatverktyg,
- kommandon som ställts till **GitHub CoPilot** och vilket svar som gavs.

Redovisning

Upplägg

Det finns fyra tillfällen under torsdag–fredag 15–16/1 att boka in sig på (torsdagen 10–13 och 15–18, samt fredagen 10–13 och 15–18). Totalt tolv pass per tillfälle. Vi kommer att ta pauser där det naturligt uppstår.

Ni får själva boka in tillfälle och tid som passar er. Om ni har väldigt specifika önskemål och någon annan bokat en plats som ni önskade får ni själva kontakta den gruppen. Ni stannar under hela det tillfället som ni redovisar på. Om ni exempelvis har redovisning torsdag 15/1 kl. 10 stannar ni ändå till 13. [\[4\]](#)

Alla i gruppen ska vara närvarande och delaktiga.

Presentation

1. UPPLÄGG OCH TIDSÅTGÅNG

- Gruppens presentation, max **10 minuter**.
- Lärares och kurskamraters kommentarer och feedback, max **5 minuter**.

2. INNEHÅLL

Samtliga ska vara delaktiga vid någon del av redovisningen. Ni ska:

1. Visa webbsidan.
 - a. Visa hur ni uppfyller kraven rörande [Gränssnitt](#).
 - b. Visa att ni kan utföra samtliga CRUD-operationer (visa, lägga till, ändra och ta bort) på er webbsida.
 - c. Kort beskriver val av CSS-ramverk och motiverar ert val.
2. Kort förklara hur ni tillämpat [Tekniska detaljer](#). Ni får visa kod, men det går också bra att bara beskriva era lösningar.
3. Reflektera över gruppens erfarenheter av arbetet
 - a. Saker ni tyckt varit roligt.
 - b. Svårigheter och utmaningar.
 - c. Om ni vill: Kod eller lösningar som ni tyckte blev extra bra och vill framhäva.

⁴ Undantag kan göras om giltiga skäl finnas. Lärare behöver kontaktas på förhand för överenskommelse av detta. Redovisningen är det allra viktigaste, men nästan lika viktigt är att ta del av varandras presentationer och lösningar för att få en bredare erfarenhet av arbetet. Det är också nyttigt att ge och ta del av feedback.

Notering rörande Live Server

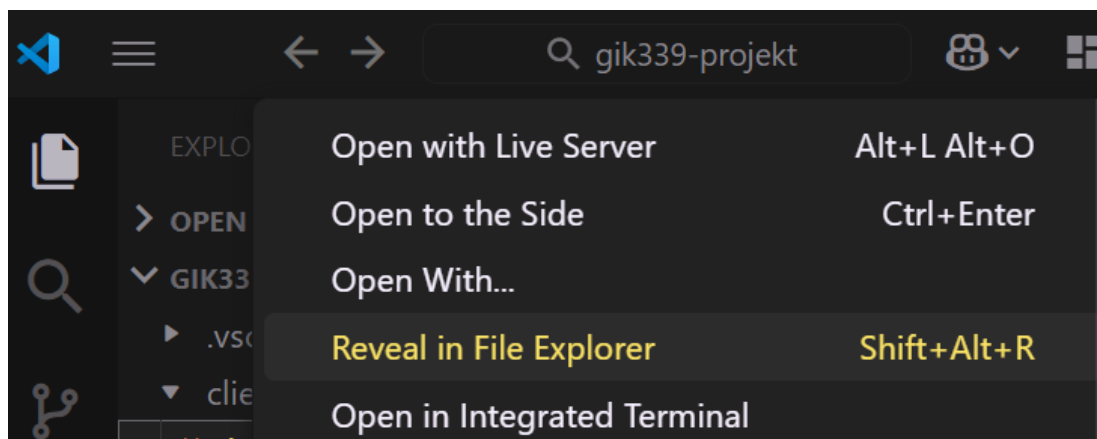
När ni använder Live Server-tillägget kommer ni eventuellt stöta på oväntade problem som bara är Live Servers fel.

Live Server är gjort för att starta om er webbsida när en förändring. Till sådana förändring räknas lagring till databas, eftersom databasfilen (den som heter något med .db) ändras. När databasfilen ändras genom att ni t.ex. sparar data kommer därför sidan att laddas om. Detta är oönskat av flera anledningar. Dels kan det göra så att eventuella meddelanden från servern inte syns eftersom sidan hinner laddas om innan de kommer fram. Det kan också göra att eventuella fel rörande att listan ska uppdateras dynamiskt döljs. Det konkreta kravet "[6 När svar från servern har mottagits ska listan uppdateras dynamiskt, utan sidomladdning.](#)" Under [Frontend Steg 6](#) ovan kan då inte testas eftersom Live Server orsakar att sidan laddas om och ni förmodligen gör en initial upphämtning av data när sidan först laddas.

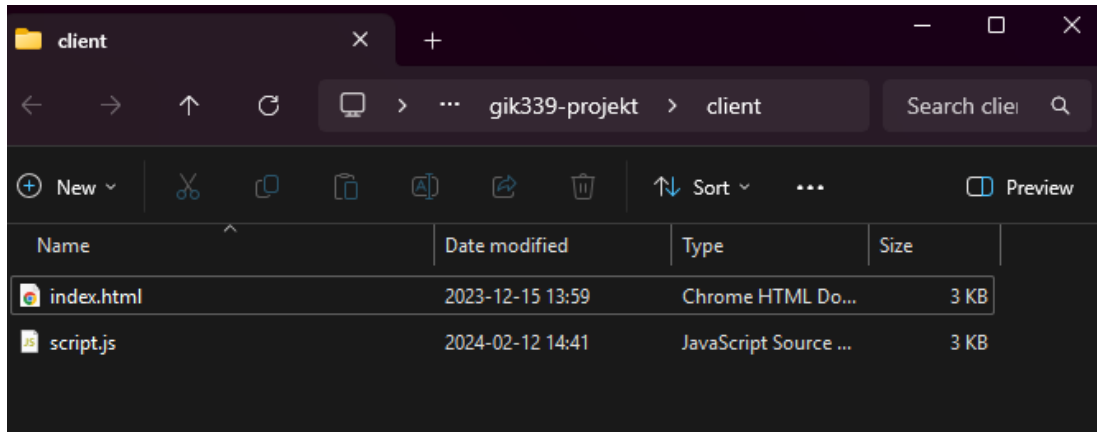
Det är smidigt att använda Live Server under utvecklingen, eftersom ni inte behöver starta om er webbsida vid förändringar, men se till att i alla fall inför inlämning och redovisning har testat utan Live Server så att ni kan verifiera alla krav. Det kan också vara en möjlig lösning ifall ni saknar svar som ni vet bör komma från backend, alternativt att svar från backend bara snabbt blinkar till på skärmen eller i konsolen och sedan försvinner.

För att försäkra er om att inte Live Server spökar kan ni öppna index.html direkt i webbläsaren, utan Live Server. Så ni behöver leta på var index.html ligger lagrad på dator. Ett sätt är genom **högerklick** på filen och **Reveal In File Explorer** och sedan (dubbel)klicka på filen för att öppna den i er standardwebbläsare. Se detaljerade steg nedan:

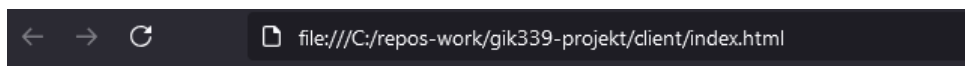
1. Leta på index.html på datorn, exempelvis genom VS Code på detta sätt:



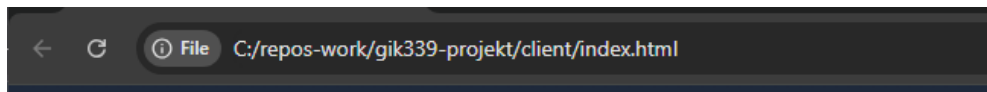
2. Dubbelklicka på filen



3. Se att adressen pekar till en fil på er dator, så att det ser ut något i stil med detta:



Eller



Så här ser adressen ut om man öppnar filen genom Live Server och det är alltså *inte det önskvärda i detta fall*:

